

Ranked by the Matcher: A Reproducibility Audit of Knowledge Graph Extraction from Threat Reports

Safayat Bin Hakim

University of Maryland, Baltimore County
Baltimore, Maryland, USA
shakim3@umbc.edu

Houbing Herbert Song

University of Maryland, Baltimore County
Baltimore, Maryland, USA
songh@umbc.edu

Abstract—Security teams and researchers choose knowledge-graph extraction tooling for threat reports on the strength of published triple-F1 scores, yet those scores depend on how predicted triples are matched to gold annotations. We could reimplement the stated matching rule for only five of twelve inspected systems. Re-scoring ten system outputs on shared documents under eight protocols reverses eleven of forty-five pairwise orderings; one fixed prediction set spans 0.16–0.70 F1. On GRID’s external 378-item calibration set, no mechanical matcher—lexical, embedding, or entailment—agrees with multi-reviewer adjudication above 71%, whereas an LLM judge reaches 86%. To separate component effects from matcher rewards, we build CTIForge, whose deterministic validation layer can vary while extraction is held byte-identical. Across seven tested deployment configurations, validation raises precision for all four hosted backbones and lowers it for all three offline backbones. Because backbone, decoding, and backend-specific prompting covary, this is a descriptive split rather than an isolated serving effect. It coincides with a roughly 2.8-fold increase in actions explicitly disputing entity type, consistent with hand-written rules encoding the conventions of the extractor against which they were developed. We release the pipeline, protocol suite, and per-triple audit records.

Index Terms—Cyber threat intelligence, knowledge graphs, neuro-symbolic AI, information extraction, MITRE ATT&CK, LLM-based extraction, symbolic validation

1. Introduction

Cyber threat intelligence (CTI) is distributed as narrative prose in vendor reports, and converting it into knowledge graphs of threat actors, malware, vulnerabilities, techniques and infrastructure is now a routine part of security operations [1], [2]. A steady stream of extraction systems has followed, most recently built on large language models, and practitioners and researchers pick among them the way one picks among any tools: by reading the numbers each paper reports.

The problem is not simply that extraction is imperfect; it is that the scores used to compare imperfect extractions may be incomparable.

Those numbers are almost always triple F1 against gold annotations. What that number means depends entirely on when a predicted triple is deemed to match an annotated one—whether entity names must be identical or merely overlap, whether relation labels must agree exactly or only be compatible, whether a reversed subject and object counts, and whether matching happens within a document or across a pooled corpus.

We audited whether the matching rules of twelve recent CTI extraction systems and benchmarks can be reconstructed from their publications (Table 1). No two inspected papers specify the same rule. Five rules were reimplementable, three named a family without fixing it, and four stated no criterion. Fieblinger et al. [3], for example, reject exact-match evaluation for generative output and adopt n -gram overlap.

The most consequential case is CTI-Nexus [4], a frequent comparison target. It counts a prediction as correct when its elements “semantically match” a gold triplet, but does not specify the encoder, threshold, or decision rule. A later paper repeating that comparison without defining its own matcher inherits a criterion it cannot reproduce.

This paper asks how much the matcher decides. Re-scoring ten published systems on a shared document set under eight protocols reverses **eleven of forty-five** pairwise orderings. Holding one fixed prediction set constant and varying nothing but the matcher moves its triplet F1 from **0.16 to 0.70**. On the external 378-item GRID calibration set [5], no mechanical protocol agrees with multi-reviewer human judgment on more than 71% of items. Pooling a corpus before matching—which several public harnesses do, including, until we found it, our own—credits a prediction from one report against a different report’s gold and inflates true positives by 6.4%.

The security consequence is direct. A defender selecting an extraction pipeline, or a researcher choosing a baseline to improve on, is acting on evidence that may not survive a change of scoring rule. The observed protocol spread can make identical predictions on identical documents look like outputs from different systems.

A question published artifacts cannot answer. Auditing released outputs bounds what comparisons mean, but it cannot say what any individual design choice contributes, because the choice cannot be switched off in someone else’s

artifact. We therefore build **CTIForge**, a five-stage neuro-symbolic pipeline whose deterministic validation layer—type-pair constraints, indicator normalization, evidence-alignment filtering, with every action logged per triple—can be enabled or disabled while extraction is held byte-identical (Fig. 1). CTIForge is apparatus for this paper rather than its proposal, and it is released as such.

Used that way, CTIForge yields an unexpected result. Across seven tested configurations, validation’s precision effect **changes sign**: it raises precision for all four hosted backbones and lowers it for all three offline backbones, with no overlap. This grouping is observational rather than an isolated serving effect because backbone, decoding, and backend-specific prompting vary together.

The split is not ordered by unvalidated F1 or listed parameter count. Instead, it tracks how much of the validator’s work disputes entity *types*: loose typing can trigger constraints on triples that are substantively correct. Because the rules were developed against frontier-model output, they encode its conventions. Deterministic does not imply model-agnostic, in this pipeline or in neuro-symbolic pipelines generally. Canonicalization, meanwhile, rewrites a small share of triples that the metric does not register.

This paper makes four contributions. First, we provide a **reproducibility audit** that re-scores ten system outputs on shared documents under a common protocol suite. Second, we **calibrate mechanical protocols** against an external multi-reviewer set rather than assuming that semantic similarity tracks judgment. Third, we give a **shared-extraction ablation** of deterministic validation and canonicalization across seven deployment configurations, isolating each downstream module’s effect conditional on fixed predictions. Fourth, we **measure two correctable evaluation defects**: corpus-pooled matching and comparative use of metrics defined by a participant’s own schema. We release the code, configurations, logs, and per-triple audit records (Appendix G).

The implementation, the evaluation harnesses, and the raw error-taxonomy logs underlying Section 6.3 are publicly available. The per-triple validation records reported here can be inspected directly.

2. Related Work

2.1. Evaluating CTI Extraction

Benchmarks for CTI language tasks have grown quickly—CTIBench [6] and SEvenLLM [7] both evaluate LLMs across CTI abilities—but graph-construction results still rely almost exclusively on triple F1 against gold annotations. Because the matching rule is often informal or absent, published figures are not directly comparable. Table 1 groups twelve systems and benchmarks by how completely we could reconstruct that rule: five were reimplementable from the paper, three left an encoder or threshold unfixed, and four stated no criterion. Seven publications therefore leave unresolved choices about name normalization, relation compatibility, or document-level scoping.

GRID [5] is the closest prior work to treat the instrument itself as an evaluation object, calibrating an LLM judge

against multi-reviewer human adjudication. We use that calibration set to evaluate ten mechanical protocols and extend the question to ordering sensitivity across eight protocols and ten system outputs. We classified each publication by reading its evaluation section and searching for a definition of when a prediction counts as correct; where none was found, we record that rather than infer one. This was one coder and one pass, with no inter-rater statistic [8]; criteria stated only in released code may have been missed. The counts characterize these inspected publications, not all CTI extraction systems.

Re-evaluation as a contribution. Security research has an established line of work that treats evaluation practice as the object of study rather than the means. Olszewski et al. [8] measure the problem at field scale, finding across nearly 750 machine-learning security papers that 60% publish no code to run their experiments; Arp et al. [9] catalog recurring pitfalls that inflate reported performance; and TESSERACT [10] shows that removing temporal bias changes malware-classification conclusions. Narrower re-evaluations find that graph-based intrusion-detection results largely fail to reproduce [11], dataset duplication and untuned baselines explain much of the reported advantage in static Android malware detection [12], and most of 38 surveyed DGA papers rest on a fragile practicality basis [13]. In CTI, Schlette et al. [14] define and visualize quality dimensions, Yang et al. [15] assess feed trustworthiness, and Geras and Schreck [16] examine operational quality assurance. That work asks whether the intelligence is good; we ask whether the numbers used to compare the systems that build it are.

The same instrument problem appears outside security. Wadhwa et al. [17] abandon exact matching for human evaluation because generative models express correct relations in forms the metric rejects. Swarup et al. [18] similarly find that extraction quality is not robust to the data encountered, while GenRES [19] proposes a multidimensional alternative to precision and recall against annotated references. Tan et al. [20] locate a complementary failure in the gold data: re-annotating 4,053 DocRED documents recovers relations the original scheme missed. SQC-Score [21] addresses both problems by refining the match and enriching the gold set with natural-language inference. We test entailment in the opposite direction—as a matcher against existing gold rather than a means of extending it—and find that it adds no value there (Section 6.1); this does not bear on the enrichment task. Flood et al. [22] likewise show in security that auditing an evaluation artifact can itself be a contribution.

We contribute the CTI knowledge-graph case, where the bias is neither temporal nor sampling-related nor a property of the data, but resides in the matching rule—an artifact of reporting rather than of experimental design, and therefore correctable at no cost. Concurrent work on ATT&CK technique extraction reports a similar sensitivity to evaluation setup [23], [24].

2.2. CTI Extraction Systems

Extraction from threat reports has moved through three generations, surveyed in depth by Rahman et al. [25] and, for

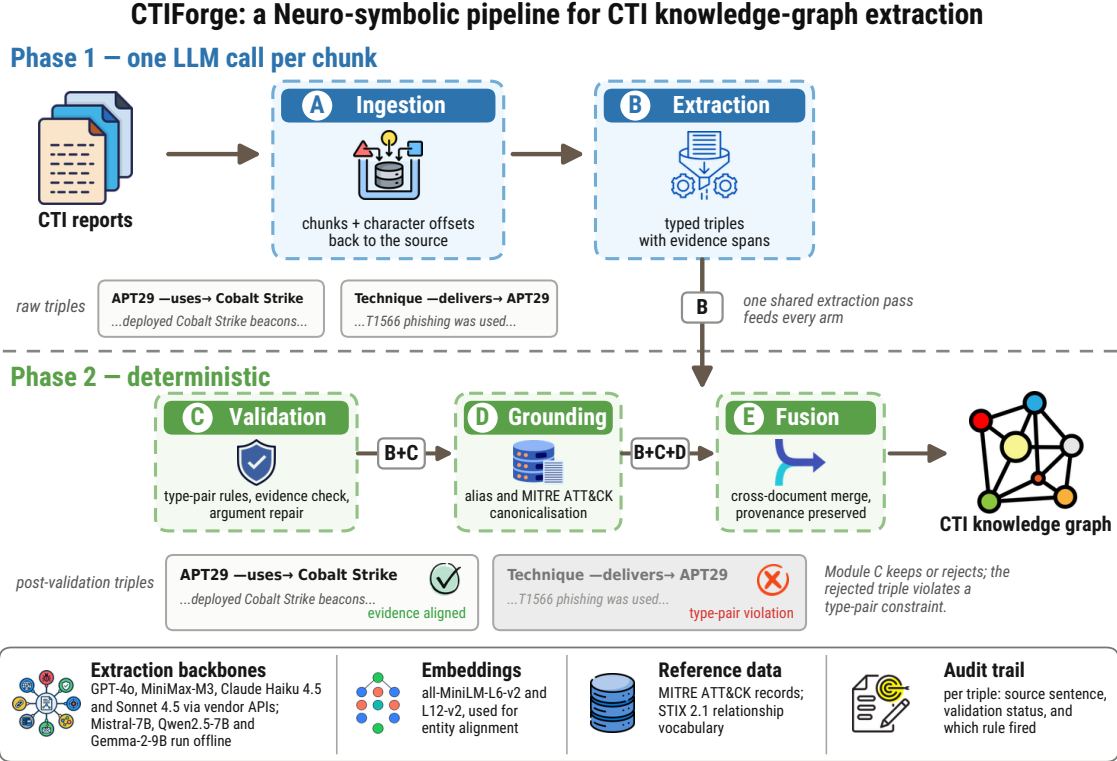


Figure 1: CTIForge pipeline architecture. Extraction is a single LLM call per chunk; every stage after it is deterministic, which is what allows all ablation arms to be derived from one extraction pass.

relation extraction, systematized by Arikkat et al. [26]. Those works catalog the methods; we audit the measurement used to rank them. Annotated corpora such as MalwareTextDB [27] established the gold-set convention used by later systems. Rule-based systems such as TTPDrill [28] offered interpretability without adaptability. Supervised neural pipelines—TIMiner [29], Open-CyKG [30], CyberRel [31], Vulcan [32], CSKG4APT [33], KnowCTI [34], CyberEntRel [35], and AttackKG+ [36]—improved coverage at the cost of annotation dependence and brittleness to new phrasing.

In-context learning removed the annotation requirement: CTI-Nexus [4] uses demonstration retrieval and multi-phase prompting, LLM-TIKG [37] distills a smaller model from LLM-labeled data, and CTIKG [38] organizes multiple agents over long reports. A parallel line adds structure around the model through role specialization [39], ontology conformance [40], [41], [42], or explicit reasoning traces [43]. Outside security, EDC [44] makes canonicalization a named stage of LLM-based graph construction, as we do in Module D.

What these systems share is the reporting convention this paper audits. Each is evaluated by matching predicted triples against gold annotations, each reports triple F1 as the headline, and—per Table 1—most do not fix the rule that decides a match. Our concern is not with any system’s design but with whether the numbers that rank them mean what they appear to.

A recent unified re-evaluation likewise finds that custom

ontologies, datasets, and metrics make reported TTP results difficult to compare [45].

2.3. Concurrent Work (2026)

Several systems appearing while this work was in preparation converge on overlapping goals. GRID [5] trains 4B extractors with task-bank reinforcement supervision, preserves verbatim evidence anchors, and contributes a 249-article benchmark with an LLM judge calibrated to 86.0% agreement against three reviewers. TACTIC-KG [41] decomposes construction into extractor, typer, verifier and curator agents on 3B–8B models. ANCHOR [40] enforces SHACL conformance over UCO, STIX 2.1 and MALOnt on the same 149 reports used here, and TTPrint [46] anchors each candidate technique to a localized evidence window before verifying it. Outside security, Loconte et al. [47] formalize ontology-grounded post-extraction correction as a general recipe. Related work also considers post-extraction provenance and corroboration (TRACE-CTI [48]) and multi-agent CTI graph construction (KGAgent4CTI [49]).

Explicit symbolic validation is therefore no longer distinctive. What remains open, and what this paper measures, is whether the evaluation used to compare any of these systems supports the comparison.

2.4. Positioning of CTIForge

Prior work largely optimizes the neural side: CTI-Nexus [4] improves in-context prompts, CTIExpert [39]

TABLE 1: Matching criteria reported by twelve CTI extraction systems and benchmarks. ✓ marks a rule reimplementable from the publication; × marks one that is not. Shaded rows are systems also re-scored under eight protocols in Section 6.1.

System	Reported matching criterion	Reprod.
<i>Rule specified precisely enough to reimplement</i>		
CTIBench [6]	exact match on named entities	✓
GRID [5]	explicit rule set, LLM judge	✓
TACTIC-KG [41]	embedding similarity, $T=0.6$	✓
ANCHOR [40]	exact match plus partial credit	✓
Fieblinger et al. [3]	ROUGE n -gram overlap	✓
<i>Similarity family named, encoder or threshold unfixed</i>		
CTI-Nexus [4]	“semantically match” (undefined)	×
LLM-TIKG [37]	cosine similarity, threshold unstated	×
SEvenLLM [7]	semantic similarity + ROUGE-L	×
<i>No matching criterion located</i>		
CTIKG [38]	not stated	×
AttacKG+ [36]	not stated	×
CTIExpert [39]	not stated	×
TTPrint [46]	not stated	×

arbitrates between specialized models, and CTIKG [38] orchestrates long-context reasoning across agents. Concurrent 2026 systems add structural control by different means—SHACL conformance in ANCHOR [40], a learned verifier in TACTIC-KG [41], span anchoring in TTPrint [46].

CTIForge occupies a specific point in that space. Extraction is a single standard LLM call, followed by a validation layer that is *deterministic* (the same triple always triggers the same rule), *categorized* (every action maps to one of a fixed taxonomy), and *logged per triple* (the rule that fired is recoverable from the artifact). Among the systems and released artifacts we inspected, none exposes all three properties together. That combination is what makes the error analysis in Section 6.3 possible: a learned verifier label alone does not expose which constraint produced it.

3. Problem Formulation: Scoring as a Free Parameter

Let $D = \{d_1, \dots, d_n\}$ be CTI documents. A system emits, per document, a set of predicted triples $\hat{Y}_d = \{(e_i, r, e_j)\}$ with e_i, e_j typed entity mentions and r a relation; a benchmark supplies gold triples Y_d . Each predicted triple additionally carries an evidence span s , a source identifier, and a validation status $\nu \in \{\text{raw, validated, repaired, rejected}\}$.

Comparing systems requires deciding when $\hat{y}$ counts as y . We make that decision explicit. A **matching protocol** is a pair $M = (\mu, \sigma)$: a predicate $\mu(\hat{y}, y) \in \{0, 1\}$ and a scoping rule σ fixing the candidate gold set against which each prediction is tested. Given M , true positives follow from a greedy one-to-one assignment in which each gold triple is consumed at most once, and

$$F1(\hat{Y}, Y; M) = f(\mu, \sigma, \hat{Y}, Y), \quad (1)$$

which is the point of writing it this way: the reported score is a function of the protocol as much as of the predictions. Two systems are **rank-reversed** under M_1, M_2

when $F1(\hat{Y}^A; M_1) > F1(\hat{Y}^B; M_1)$ and $F1(\hat{Y}^A; M_2) < F1(\hat{Y}^B; M_2)$.

The predicate μ varies along three axes used in the literature: *name equality* (exact string, containment, stemmed-token overlap, embedding similarity above a threshold, entailment), *relation equality* (identical after normalization, drawn from a compatibility set, or ignored), and *argument order* (fixed, or swap-tolerant). The scope σ is either per-document or pooled over the corpus. Section 6.1 instantiates eight points in this space and measures the consequences; Appendix E gives each definition in full.

Knowledge-graph quality is a studied problem with an established vocabulary—accuracy, completeness, consistency, timeliness—and a literature on assessing each [50]. That vocabulary has not been adopted in CTI extraction, where a single gold-matched F1 stands in for all of it, and the substitution is what we examine. Two properties of a protocol are worth separating. It is **comparative** only if μ depends on no participant’s ontology—STIX 2.1 vocabulary compliance and evidence presence qualify, a system’s own type-pair table does not, since the system that defines the table enforces it at extraction time and cannot score poorly against it. And it is **calibrated** only to the degree that μ agrees with a human adjudicating the same pair; Section 6.1 measures that agreement rather than assuming it.

Our apparatus (Section 4) targets precision and provenance rather than recall: it rejects or repairs structurally invalid output before graph construction, preserves an evidence span and a validation status on every edge, and merges across documents without opaque coreference. It uses a compact ontology of 14 entity types and 12 relation types; the inventories (Tables B1, B2) and the type-pair constraints (Table C1) are in Appendix B.

4. Apparatus: The CTIForge Pipeline

Section 6.1 bounds what published comparisons mean, but it cannot say what any individual design choice contributes: a component in someone else’s released artifact cannot be switched off. CTIForge exists to make that possible. It is a five-stage pipeline (Fig. 1) in which each stage is independently toggleable, extraction is separable from everything downstream, and every triple reaching the graph carries a validation status, a source chunk identifier, and an evidence span traceable to the originating sentence.

One property matters for the measurements that follow. Every stage after extraction—validation, saliency filtering, canonicalization, alignment—is deterministic given its input, and link prediction, the only other stage that calls a model, runs *before* the validation toggle. All ablation arms can therefore be derived from one extraction pass, so an arm differs from another by the module under test and by nothing else. Section 6.2 depends on this.

Extraction is probabilistic; validation is deterministic; canonicalization is conservative and alias-driven rather than embedding-driven; fusion is confidence-aware and provenance-preserving. We describe each stage only to the

depth the measurements require; implementation detail is in Appendix C.

4.1. Ingestion and extraction (Modules A–B)

Reports are segmented into paragraph-level chunks with stable identifiers and character offsets back into the source, so every downstream claim retains a pointer to the sentence that produced it. Extraction is a single LLM call per chunk against a schema-constrained prompt, with optional few-shot retrieval and a second call for link prediction only when the document graph is disconnected. Keeping extraction to one call per chunk is what makes the shared-extraction ablation cheap enough to run on seven backbones.

4.2. Symbolic validation and repair (Module C)

Module C is the component this paper measures, so we state it precisely. Each predicted triple is treated as a candidate that must pass a deterministic sequence of checks before graph construction: empty-field rejection; indicator-format repair; type repair for *Other*; type-pair validation against a fixed constraint table, with an auto-swap pass for reversed arguments; self-loop rejection; evidence alignment against the cited span; placeholder detection for mentions such as “the attacker”; and ATT&CK identifier validation. Each check emits one of three actions—*rejected*, *repaired*, or *flagged*—into a per-triple log tagged with one of fourteen error categories, so the rule that admitted or altered any edge is recoverable from the artifact rather than from the code.

A separate conservative repair step promotes weak *related_to* relations to specific types when the evidence supports it, using keyword matching against eleven relation categories. It does not promote *associated_with* by default and requires both entity mentions to appear in the evidence span before acting.

Two properties of this design are load-bearing later. The checks are hand-written rules with fixed thresholds, developed against the output of a strong extraction model—Section 6.2 shows what happens when they meet a weaker one. And they are deterministic, which is what allows an ablation arm to be recomputed from stored extraction output rather than re-extracted.

4.3. Grounding, fusion, and optional stages (Modules D–E)

Module D reduces surface fragmentation in three passes: alias-based canonicalization over a curated table of known actors, malware, and tools; type-gated embedding alignment at a conservative threshold ($\tau = 0.85$), so a malware name is never merged with an organization; and ATT&CK technique grounding, which attaches an identifier as provenance rather than rewriting the surface form. Module E merges per-document graphs while preserving per-triple origin and confidence. Three optional stages—link prediction, embedding entity alignment, and saliency filtering—are retained; three others were implemented and disabled after they over-generated on CTI text (Appendix G).

Because canonicalization rewrites strings that gold annotations record verbatim, it can improve graph usability while costing gold-matched F1. Section 6.2 measures that trade and finds the metric registers neither gain nor loss.

5. Experimental Design

We first vary the matching protocol to measure evaluation sensitivity, then hold extraction fixed to isolate validation and canonicalization, analyze the resulting error categories, compare systems under uniform criteria, and finally examine backbone and runtime limits.

5.1. Datasets and Resources

Table G1 summarizes the datasets and external resources used in evaluation.

CTI-Nexus benchmark. The primary evaluation corpus consists of 149 gold-annotated CTI reports from the CTI-Nexus dataset [4]. Each report contains typed entities and explicit triples in JSON format. An official test split of 11 documents is used for direct comparison with CTI-Nexus reported metrics; the remaining 138 documents serve as the few-shot example pool.

CTIKG benchmark. The secondary benchmark consists of 255 CTI sentences with 693 gold triples from the CTIKG dataset [33]. Each sentence is annotated with bracket-format triples and tagged with its ATT&CK tactic. Crucially, CTIKG’s own predicted triples are included in the dataset, enabling head-to-head comparison under identical evaluation criteria.

Train/test separation. For CTI-Nexus evaluation, few-shot examples are drawn exclusively from the training split. The retriever excludes the current document during example selection to prevent trivial leakage.

5.2. Research Questions

- 1) **RQ1:** How much of a reported result in this literature is decided by the matching protocol rather than by the system?
- 2) **RQ2:** What do symbolic validation (Module C) and canonicalization (Module D) contribute when extraction is held fixed?
- 3) **RQ3:** What is the distribution of errors across the symbolic error taxonomy?
- 4) **RQ4:** How does CTIForge compare with prior systems under uniform criteria and at matched backbone?
- 5) **RQ5:** What limits further improvement, and how much of that limit is the extraction model?
- 6) **RQ6:** What does CTIForge cost to run relative to comparable pipelines?

5.3. Evaluation Metrics

Matching is scoped *per document*. Pooling every document’s predictions and gold into flat lists before matching permits a prediction from one report to be credited against gold from another; on our data that inflates true positives by 6.4%. All reported figures use per-document scoping.

We report three families, and keep them separate because they license different claims. **Gold-matched** metrics (subject–relation–object; relation-free subject–object pair) compare against annotations under a stated matcher. Neither is a strict-equality match: both apply soft name matching, and the triple metric additionally accepts a fixed relation-compatibility set, so both should be read as *soft* protocols throughout. **Schema-independent** metrics—evidence presence, duplicate rate, and compliance with the external STIX 2.1 relationship vocabulary—depend on no participant’s ontology and are therefore valid for cross-system comparison. **Schema-dependent** metrics scored against our own type-pair table are *definitional*: the validator enforces that table at extraction time, so CTIForge cannot score poorly on them. We report them in Appendix I as evidence that the layer enforces its contract, never as evidence of superiority over another system.

Matcher definitions, normalization rules, and the type-equivalence groups are given in Appendix E.

5.4. Experimental Configuration

All experiments use a precision-first configuration unless otherwise noted. GPT-4o is the default backbone for the full 149-document CTI-Nexus run and the matched-backbone comparison, using OpenRouter [51] for both systems. The 20-document development slice uses Mistral Small 3.1 24B. The substitution study in Section 6.5 adds MiniMax-M3, Claude Haiku 4.5, and Sonnet 4.5 through hosted APIs, plus Mistral-7B-Instruct, Qwen2.5-7B-Instruct, and Gemma-2-9B offline on one consumer GPU.

All post-extraction stages are identical and deterministic. Hosted extraction uses temperature 0.1; offline extraction is greedy and adds a local JSON-format wrapper to the common prompt. Backbone, serving stack, decoding, and wrapper therefore covary, so this study measures transfer across deployment configurations rather than an isolated serving effect. We excluded two candidate hosted backbones whose temperature settings could not be matched, as discussed in Section 7.

Validation keeps evidence rejection enabled, canonicalization uses an alignment threshold of $\tau = 0.85$, and the retained enhancements are link prediction, embedding-based entity alignment, and saliency filtering. Every configuration is released; Appendix G gives the full hyperparameter table.

5.5. Ablation Design

The module-level ablation isolates symbolic validation and canonicalization over the full benchmark. Extraction is run *once* and all arms derive from the same raw triples; every stage downstream is deterministic, so arms differ only by the module under test. Running each arm end-to-end instead would confound the module with a fresh sampling of the extraction model, and at our operating temperature that resampling noise is comparable to the effect being measured. The enhancement-level ablation, which toggles the optional neuro-symbolic additions around B+C+D, remains in Appendix G.

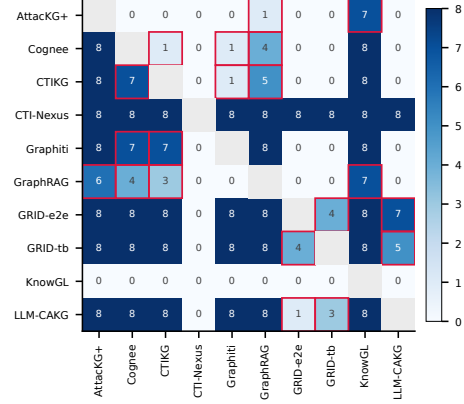


Figure 2: Pairwise orderings among ten published systems under eight matching protocols. Each cell counts the protocols in which the row system outranks the column system; shading tracks that count. Outlined cells are the eleven pairs whose ordering is not stable across protocols.

6. Results and Analysis

We report the measurement question first. Every subsequent number in this section is produced by a matching protocol, and Section 6.1 establishes how much that choice decides; the results that follow should be read against that bound rather than at face value.

6.1. RQ1: How much does the matching protocol decide?

Every number above is produced by one matching protocol. Two measurements bound how much that choice decides.

Finding 1: Mechanical protocols leave a substantial gap on GRID’s human-adjudicated set.

We score ten protocols—exact match, three soft-lexical variants including ours, three embedding thresholds, and three natural-language-inference thresholds—against the 378 adjudicated items of the GRID judge-calibration set [5], in which three reviewers ruled on whether an edge matches a reference graph. Agreement spans 0.598 to 0.706 (Fig. 3a), and the dominant failure is *under-matching*: the strict protocol misses 40.2% of the matches a human credits. Part of that is the matcher being too strict, and part is gold annotation being incomplete in the way Tan et al. [20] document for DocRED—a correct prediction with no annotated counterpart is indistinguishable from a wrong one. *No mechanical protocol exceeds 0.71*, including the entailment family, which we added specifically to test whether a semantic matcher escapes the ceiling; it does not. An LLM judge reaches 0.860 on the same items. With 378 items the resolution is about ± 0.05 , so the mechanical protocols should be read as one group rather than as a ranking; the gap to the judge is several times that width. The instrument the field reports therefore disagrees with human adjudication on roughly three items in ten. We do not read the judge’s margin as a recommendation to adopt judges uncritically: they carry

documented biases of their own, including position, verbosity, and self-preference effects, and human evaluators are not bias-free either [52]. Nor does a judge’s margin transfer across settings. Laskar et al. [53] benchmark eight LLM judges over three biomedical relation-extraction datasets and find them below 50% accuracy, largely because extracted relations follow no standard format. That condition holds, notably, for several of the systems in Table 1. The comparison establishes only that mechanical matchers leave a large gap, not that any particular replacement closes it.

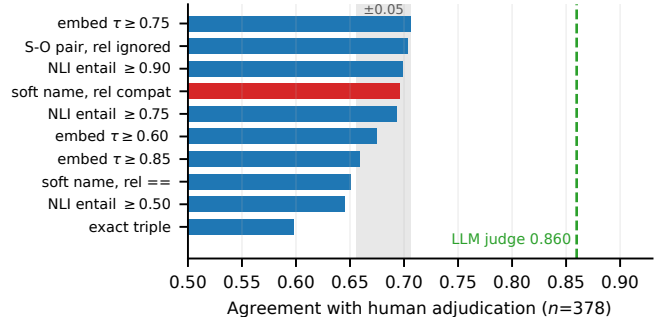
Finding 2: A system’s rank is substantially a property of the matcher. We score ten published systems—CTI-Nexus, CTIKG, AttacKG+, KnowGL, GraphRAG, Graphiti, Cognee, LLM-CAKG, and two GRID variants—on 50 shared documents under the eight protocols.

Provenance of these predictions. We did not re-run these systems. We use the prediction cache released with GRID [5]. Two criteria motivated that choice. First, *uniform backbone*: every system in the cache was run on the same extraction model, so backbone is held constant and the matcher is the only variable. Second, *shared documents*: all 50 articles are a verified subset of our 149-document benchmark, so the gold annotations are those used throughout this paper. Re-running ten systems ourselves would have reintroduced exactly the confound the audit is designed to remove, since each would have run on whichever backbone its authors assumed.

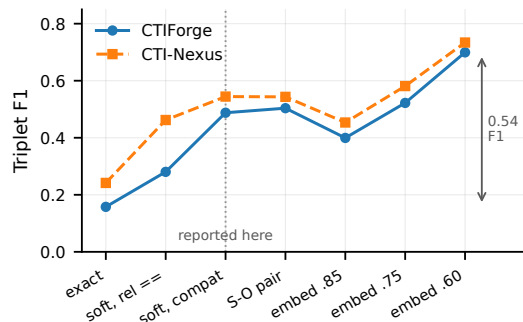
The inheritance also constrains what the comparison licenses. Because every system ran on a backbone that is not the one its authors selected, these figures are *not* evidence that any pipeline outperforms another, and we do not use them for that; they are evidence only about the sensitivity of orderings to the matcher. Coverage is 50 of our 149 documents, so the reversal count is a lower bound on what a larger shared set might show rather than an estimate of a population value. Systems absent from the cache are absent from this analysis; we did not select them out. Eight protocols over ten systems reverse **11 of the 45 pairwise orderings** (Fig. 2). The swings behind those reversals are not marginal. GraphRAG scores 0.000 under exact match and 0.414 when the relation is ignored. AttacKG+ and KnowGL are likewise zero under one protocol and non-trivial under another, for the same reason: exact string equality is unreachable for a system that emits free-text relations. A system’s rank here is substantially a property of the matcher.

Finding 3: The same predictions support a fourfold range of reported F1.

Holding one CTIForge prediction set fixed—the 149-document GPT-4o run of Appendix J—and varying only the matcher across seven protocols moves its triplet F1 (Fig. 3b) from **0.1577** under exact match to **0.6994** under an embedding matcher at $\tau = 0.60$, a spread of **0.5417** on identical predictions (Fig. 3b). The protocol we report gives 0.4877. CTI-Nexus, scored the same way, spans 0.2416–0.7339. Neither ordering flips in this pair—CTI-Nexus leads under all seven—so here the protocol decides magnitude rather than rank; the ten-system comparison above shows where it decides rank as well. The spread is not caused by



(a) Agreement with human adjudication on the 378-item GRID set [5], ours in red; the band is the ± 0.05 resolution at this sample size.



(b) Triplet F1 of one fixed prediction set under seven of those protocols, names abbreviated from (a). The dotted line marks the protocol we report.

Figure 3: The matching protocol as a free parameter. No mechanical protocol reaches the LLM judge, whose figure is the rate at which reviewers agreed with it in [5] rather than an independent re-measurement, and the choice among them moves reported F1 by more than half a point.

any system behaving badly. It follows from three choices that the protocols in Table 1 leave unfixed: whether entity names must be identical or may overlap, whether relation labels must match exactly or merely be compatible, and whether matching is scoped within a document or pooled. Each is a reporting decision rather than an experimental one, and each is invisible in a results table. A reader comparing a published 0.16 against a published 0.70 would infer a different system, not a different set of three decisions.

We place this first because it bounds every comparative claim that follows, including our own. An instrument that disagrees with human adjudication three times in ten on the external calibration set and reorders a quarter of all system pairs in our cache does not support interpreting small differences in gold-matched F1 as evidence about a system. The remaining questions are answered with that limit in view; Section 7 returns to what should be reported instead.

6.2. RQ2: What does the symbolic layer contribute?

Table 2 reports the full-pipeline results on all 149 CTI-Nexus-annotated reports (1,824 gold triples), using GPT-4o

TABLE 2: Full-pipeline results on the CTI-Nexus benchmark (149 reports, 1,824 gold triples). Triple matching compares subject, relation, and object under the three-pass protocol of Appendix E; entity types are not compared. S-O pair ignores the relation.

Metric	Precision	Recall	F1
Triple (S-R-O)	0.5331	0.4282	0.4749
S-O pair (relation-free)	0.5959	0.4786	0.5309
Entity type classification	0.5479 accuracy / 0.5082 micro-F1		
Predicted triples	1,465		
Processing time	1,586.2 s total / 10.6 s per document		

as the extraction backbone and the enhancement flags from Appendix Table G3.

Two observations structure the rest of the results. First, S-O pair F1 exceeds triple F1 by a wide margin, so the system identifies the participants of most relations and the residual gap is concentrated in relation normalization rather than entity recognition. Second, the pipeline emits fewer candidates than there are gold triples, an intentional consequence of the precision-first configuration that frames the recall ceiling of Section 6.5.

Finding 4: Validation’s observed effect separates the tested hosted and offline configurations.

A validation layer is expected to trade recall for precision by discarding bad output. Across the seven tested configurations it does so for all four hosted backbones and for none of the three run offline on our own hardware. Table 3 reports three arms per backbone, each derived from one shared extraction pass; the precision column changes sign at that observed boundary, and the groups do not overlap on any column. Because serving mode is not varied independently of backbone, decoding, and backend-specific prompting, the boundary is descriptive rather than causal. It is not ordered by unvalidated F1—the hosted model with the lowest extraction F1 still gains, while an offline model that outscores it still loses—or by parameter count within the offline group, where the largest model falls between the two smaller ones. Seven configurations are an observation, not a law, and two hosted changes are too slight to carry weight alone; the defensible statement is that no tested hosted backbone lost precision and every tested offline backbone did.

Finding 5: The inversion is consistent with a schema-conformance mismatch.

We classify *type-misassignment* and *impossible-type-pair* actions as those that explicitly dispute entity typing; structural and placeholder actions are kept outside that category. Their share differs sharply between the tested configuration groups (Figure 4): roughly one action in eight for hosted backbones and one in three for offline backbones, without overlap. This association is consistent with a schema-conformance mismatch. Type-pair constraints are the sharpest instrument in the cascade, since an impossible pair is rejected outright rather than flagged, so a backbone that types its entities loosely can trip the rule on facts that are substantively correct. Our type-pair table, evidence thresholds, and placeholder

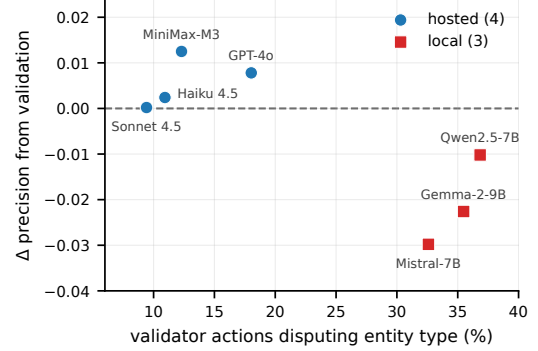


Figure 4: Validation-induced precision change against the share of actions disputing entity type, one point per backbone. The dashed line marks no change. Deployment groups are descriptive because backbone, decoding, and serving stack covary.

TABLE 3: Module ablation over seven backbones. Within each backbone, all arms derive from one extraction pass. ΔP is B+C minus B; *collateral* is the share of removals that discard a correct triple, and *type* is the share of actions disputing entity type.

Backbone	Serving	B F1	B+C F1	ΔP	Collat.	Type
MiniMax-M3	hosted	0.4440	0.4371	+0.0125	31%	12%
GPT-4o	hosted	0.5076	0.4907	+0.0078	46%	18%
Haiku 4.5	hosted	0.4418	0.4255	+0.0024	43%	11%
Sonnet 4.5	hosted	0.4239	0.4075	+0.0002	47%	9%
Qwen2.5-7B	local	0.4104	0.3706	-0.0102	55%	37%
Gemma-2-9B	local	0.4305	0.3834	-0.0226	69%	35%
Mistral-7B	local	0.4255	0.3663	-0.0298	69%	33%

heuristics were developed against frontier-model output and presume a backbone that respects the ontology it was given. A smaller model can produce triples that are structurally noisier without being less accurate—raw precision on the offline backbones is comparable to the hosted ones, and on two of them higher (Appendix Table F1)—and the rules can reject the noise together with the signal. The symbolic half of a neuro-symbolic pipeline is not model-agnostic merely because it is deterministic; it inherits the conventions of whatever produced the text it reads.

Finding 6: The metric cannot see canonicalization at all. B+C+D scores identically to B+C to four decimals on every backbone, yet on MiniMax-M3 the two prediction sets differ on 54 of 1,571 triples. Module D rewrites surface forms—alias resolution, casing, ATT&CK technique names—and the matcher’s soft name comparison absorbs every rewrite. The metric is not judging canonicalization to be neutral; it cannot see it.

Section 6.1 licenses reading these deficits at different weights: under an instrument this protocol-dependent, a third-decimal F1 change is not evidence either way, whereas a sign flip in precision accompanied by a doubling in the share of removals that were correct is.

TABLE 4: Error taxonomy distribution. Counts are validator actions, not triples; a single triple may generate more than one action. Seven of the fourteen declared categories fire at benchmark scale; the remainder are listed in Appendix Table G2.

Category	CTI-Nexus (149 docs)	CTIKG (255 sent.)
Confidence lowered	716 (67.0%)	173 (31.5%)
Impossible type-pair	104 (9.7%)	72 (13.1%)
Repaired identifier	104 (9.7%)	23 (4.2%)
<i>direction swap</i>	58	12
<i>identifier/relation</i>	46	11
Type misassignment	75 (7.0%)	162 (29.5%)
Generic placeholder	38 (3.6%)	96 (17.5%)
Self-loop	17 (1.6%)	12 (2.2%)
Hallucinated entity	15 (1.4%)	12 (2.2%)
Total actions	1,069	550
Sources touched	146 / 149	193 / 255
<i>By action</i>		
Flagged	843 (78.9%)	443 (80.5%)
Rejected	122 (11.4%)	84 (15.3%)
Repaired	104 (9.7%)	23 (4.2%)

6.3. RQ3: How are errors distributed across the taxonomy?

Every validator action is written to a JSONL log tagged with one of the taxonomy categories introduced in Appendix Table G2. Table 4 reports the distribution on both benchmarks. The profile is configuration-dependent, but that dependence is itself informative rather than a reason to omit the table, so we report both benchmarks and explain the difference.

Four findings follow. First, the validator is predominantly a *graded annotator rather than a hard gate*: 78.9% and 80.5% of its actions are soft flags, against 11.4% and 15.3% outright rejections. Confidence-lowering dominates because the evidence span an LLM cites typically supports a predicted triple partially rather than fully; retaining such triples at reduced confidence preserves graph utility that a binary accept/reject classifier would discard. This is a deliberate design point, and we state it plainly because the term “validation” invites the opposite reading.

Second, the profile shifts substantially between benchmarks, and the shift tracks input length. Long CTI-Nexus reports are dominated by confidence-lowering (67.0%), where partial evidence alignment is the recurring failure. Short CTIKG sentences spread across type misassignment (29.5%) and generic placeholders (17.5%), because a single sentence offers little context for typing and frequently refers to entities pronominally. A practitioner porting this validator to a new corpus should expect the dominant category to follow the input granularity rather than remain fixed.

Third, only seven of the fourteen declared categories fire on either benchmark; the remainder guard conditions the parser or alias tables resolve earlier, or are declared but unreached. Appendix Table G2 gives the full set. We report the observed distribution, not the declared one.

Finding 7: The error taxonomy is a census of actions taken, not of errors present. An error taxonomy invites reading as a complete accounting of what went wrong. It is not, and the gap runs in both directions. The *repaired*

identifier label aggregates three mechanisms, and 58 of its 104 CTI-Nexus actions are subject–object direction swaps rather than identifier normalization (12 of 23 on CTIKG), so we give the split. Type checking is likewise partial by construction: the constraint table rejects pairs that are *impossible*, not merely wrong, flagging 75 misassignments against 600 entities whose type disagrees with gold—roughly one in eight. The layer’s guarantee is provenance, not completeness, and a reader taking Table 4 for a full error census would overestimate both.

6.4. RQ4: How does CTIForge compare with prior systems?

Scored under one matcher rather than each system’s own, CTIForge and CTIKG sit at opposite points of the same curve: CTIForge is more precise (0.5722 against 0.5122) on roughly 57% as many triples, CTIKG reaches the higher F1 (0.590 against 0.501) on recall. Which point is preferable depends on the downstream task. The instructive number is not either of those: CTIKG’s own reported precision, 0.9189, exceeds its uniform-protocol precision by more than 35 points, entirely because its published matcher is more lenient than ours. Appendix L gives the full comparison.

Matched-backbone comparison. Removing the backbone confound is expected to settle which pipeline is better. It does not, however, and the reason is instructive. Re-running both systems on all 149 documents with the same underlying model leaves CTI-Nexus ahead by 0.057 triplet F1 (0.4877 vs. 0.5449), driven entirely by recall—it emits 2,092 triples against 1,436—while CTIForge holds a 4.4-point precision advantage. Following Section 6.1 we do not read a gap of that size under a single protocol as a ranking, which is also why the full comparison is reported in Appendix J rather than here. The margins that survive the caveat are the schema-independent ones: under STIX 2.1, an external vocabulary neither system enforces, CTIForge reaches 0.705 relationship compliance against 0.573, and every emitted triple carries a source sentence because the `Triple` schema requires one, whereas CTI-Nexus emits no evidence field at all. That is a contract difference no choice of matcher affects.

6.5. RQ5: What limits further improvement?

Reading the full-pipeline numbers in Table 2 and the same-backbone comparison in Table J1 together identifies three concrete bottlenecks separating the current pipeline from higher end-to-end triplet F1.

Bottleneck 1: relation labeling, not entity identification. S-O-pair F1 reaches 0.5309 while triplet F1 is 0.4749. The pipeline identifies the participants of a relation more reliably than it labels the relation. Two mechanisms contribute: CTI-Nexus gold uses free-form relation strings that must be normalized into the 12-relation inventory, and a nontrivial fraction collapse into the catch-all types; and semantically adjacent predictions (*uses* vs. *exploits*, *delivers* vs. *drops*) are not consistently disambiguated. The effect is far larger on CTIKG, whose free-text gold drives strict

F1 to 0.0568—a property of the ontology mapping, not of relation quality, and the reason we report the relaxed match as the headline there. The residual gap to CTI-Nexus at matched backbone is likewise recall-dominated, and is the combined effect of saliency filtering, evidence rejection, and cross-chunk deduplication suppressing marginal triples by design.

Bottleneck 2: entity typing is the weakest local component. Accuracy is 0.5479. Errors concentrate on recurring borderline cases—programming languages tagged Tool vs. Software, threat-actor names tagged Organization vs. ThreatActor—and ontology-guided assignment reports substantially higher figures on this corpus [40].

Finding 8: precision is backbone-insensitive; recall is not. A weaker extractor might be expected to degrade the pipeline uniformly, but the two quantities separate across the seven tested backbones (Table 3). Unvalidated precision spans only eight points, with all three offline backbones in the upper half (Appendix Table F1). Recall separates the groups more sharply: the offline models occupy the bottom and lie within one point of one another.

Figure F1 puts this difference in proportion. Within any backbone, the interquartile spread of per-document F1 is several times the gap between hosted and offline group means. Recall also varies more widely among hosted models, suggesting that the pipeline’s saliency cap, evidence rejection, deduplication, and per-chunk budget become limiting in that regime. The tested free-tier backbone reproduces the paper’s qualitative conclusions at no cost, while a fully offline configuration remains viable at roughly 0.11 F1 below the strongest tested configuration.

6.6. RQ6: What does the pipeline cost to run?

Finding 9: the symbolic layer is not what costs time. On the identical benchmark, backbone, and endpoint, CTIForge completes in roughly two-thirds of CTI-Nexus’s wall-clock time. The saving is architectural: CTIForge makes one LLM call per chunk, whereas CTI-Nexus uses a multi-phase prompting pipeline. Validation, canonicalization, and fusion run locally at negligible cost, which also enables the shared-extraction ablation. Appendix K gives the timings.

7. Discussion

The strongest supported result is that evaluation protocol changes both score magnitude and, in some comparisons, system ordering; the component findings below should therefore be read as conditional measurements rather than a new universal ranking.

What the symbolic layer actually does. Table 4 corrects a natural misreading of “validation”: roughly four in five logged actions are *soft flags*. The layer is therefore predominantly a graded annotator that records why a triple is doubtful, not a gate that discards it. Table 3 further shows that its value depends on the output being annotated: the tested hosted configurations trade recall for precision, whereas the tested offline configurations lose precision and mostly remove correct triples.

For analyst-facing graphs, this result favors grading over gating because partially correct triples may retain useful structure. The layer does not guarantee that surviving triples are correct; it guarantees a source sentence, validation status, and rule record for each one. ANCHOR [40] enforces conformance through SHACL, TACTIC-KG [41] through a learned verifier, and CRUcialG [54] through attack-rationality checks that can repair relations. Among the inspected artifacts, we found no equivalent categorized per-triple record; a verifier label alone does not identify the constraint that produced it.

Limitations. At matched backbone, CTIForge trails the recall-oriented CTI-Nexus pipeline by 0.057 triplet F1, with the gap entirely in recall. Raising the saliency cap or relaxing evidence rejection may alter that trade-off, but we do not test those interventions. Mapping free-text gold into 12 relations is also lossy, most visibly on CTIKG (strict F1 0.0568), which is why we report the relaxed match there. Canonicalization can improve graph usability while changing strict F1 because gold annotations preserve the surface forms it rewrites.

The shared-extraction design isolates downstream modules conditional on one fixed prediction set per backbone; it does not estimate between-generation variance. The near-zero Sonnet and Haiku changes are therefore descriptive, not stable effect estimates. Nor do we establish that a soft flag is calibrated: it records which rule fired, not a measured probability of correctness. Finally, ATT&CK grounding covers only Technique entities—the best tested open-source LLM reaches 0.22 micro-F1 [23]—and cross-document fusion is exercised but not benchmarked. Both require curated evaluation sets that we have not built.

Empowering downstream defenses. Because every edge carries provenance, the graph can be exported to STIX and exchanged without laundering its origin, and validation status supports triage—filter to unrepaired, evidence-aligned edges when confidence matters, admit flagged edges when coverage matters. Evidence that multi-report aggregation improves technique extraction by roughly 26% [24] suggests fusion is where the most value remains unrealized.

The sampling temperature is becoming unobservable. Matching decoding settings across hosted endpoints was routine bookkeeping until recently; it no longer is. Several frontier endpoints we screened could not be matched to the 0.1 temperature used by the retained hosted runs: one vendor’s current tier rejects the parameter as deprecated, another accepts it but applies it so weakly that repeated calls on an identical prompt reproduced under two-fifths of their own triples, and a third refuses any value but its default. We excluded those hosted backbones rather than report an arm whose sampling condition we could not state. This is the concern of Section 6.1 one layer lower—there the undeclared free parameter is how predictions are scored, here it is how they were generated—and a configuration file recording a temperature the endpoint did not honour is worse than one recording nothing.

8. Recommendations

A critique of an instrument is useful only if it says what to do differently. The following recommendations are evidence-based practices suggested by the measured defects, not guarantees that every dataset or system will behave identically. Our results support four changes, none of which requires new annotation.

8.1. For researchers reporting results

R1. Scope matching within a document. Pooling a corpus into flat prediction and gold lists before matching lets a prediction from one report be credited against a different report’s annotations. On our benchmark that inflates true positives by 6.4%, and nothing in a results table reveals it. Several public harnesses do this. Ours did too, silently, until we audited it.

R2. Report a spread, not a point. A single F1 conceals both that eleven of forty-five system orderings flip under a different protocol and that one fixed prediction set spans 0.54 F1 across protocols (Fig. 3b). Reporting a protocol spread costs one table; we give ours rather than only prescribing it.

R3. Separate metrics by what they license. Figures scored against a system’s own schema are definitional—the system enforces that schema at extraction time and cannot score poorly against it—and cannot rank systems that do not share it. Keeping them apart from schema-independent measures such as STIX 2.1 vocabulary compliance prevents a conformance result from reading as a competitive one. The knowledge-graph community already separates quality dimensions this way [50]; the recommendation is to import that discipline, not to invent it.

R4. State the matching rule precisely enough to reimplement. We could reconstruct five of the twelve inspected rules from their publications (Table 1). “Semantic matching” without an encoder and a threshold is not a specification, and a downstream comparison inherits that ambiguity unless it defines a new rule. Releasing an artifact does not substitute for stating the rule: across eleven years of applied security venues, ACSAC reaches 90% artifact participation in its evaluation committee while only 40% of those artifacts run [55]. Where exact matching is genuinely unsuited to generative output, a multi-dimensional protocol such as GenRES [19] is preferable to an unspecified similarity, because it can at least be reimplemented.

8.2. For practitioners selecting tooling

R5. Treat a single published F1 as a lower bound on uncertainty, not as a measurement. The gap between two systems in this literature is routinely smaller than the gap the same system shows across protocols. Where a procurement decision rests on such a comparison, re-score the candidates on your own documents under at least two protocols.

R6. Prefer artifacts that carry per-edge provenance. Whatever the score, an edge that cites the sentence that produced it can be checked by the analyst who must act on it; one that does not, cannot. This property is independent of the matcher and survives any change of protocol.

9. Conclusion and Future Work

CTI knowledge-graph results are not separable from the protocols that produce them. Re-scoring shows that both reported magnitude and system ordering can change with the matcher, while external human calibration exposes a substantial gap for every mechanical family tested. Matching rules should therefore be explicit, document-scoped, and reported as a spread rather than a single unexplained F1.

The shared-extraction ablation adds a second lesson: deterministic rules did not transfer uniformly across the seven deployment configurations. Their sign split and differing type-action shares are consistent with schema-convention mismatch, but do not isolate serving as its cause. *Deterministic* and *model-agnostic* are different properties.

The external calibration set bounds matcher families rather than measuring CTI triple equivalence in-domain; a multiply annotated CTI set remains necessary. We release the pipeline, configurations, scoring suite, ablation harness, and per-triple records so the reported measurements can be inspected and re-scored.

10. Ethical Considerations

Our corpora are public vendor reports and their published annotations, and the graphs the pipeline produces contain nothing not already in the source reports. We conducted no experiments involving human subjects: the adjudications used in Section 6.1 were collected and released by prior work [5] under that work’s ethical review, and we reuse them without re-identifying any reviewer. Systems we re-score are evaluated from artifacts their authors released for that purpose.

References

- [1] N. Sun, M. Ding, J. Jiang, W. Xu, X. Mo, Y. Tai, and J. Zhang, “Cyber threat intelligence mining for proactive cybersecurity defense: A survey and new perspectives,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 3, pp. 1748–1774, 2023.
- [2] T. D. Wagner, K. Mahbub, E. Palomar, and A. E. Abdallah, “Cyber threat intelligence sharing: Survey and research directions,” *Computers & Security*, vol. 87, p. 101589, 2019.
- [3] R. Fieblinger, M. T. Alam, and N. Rastogi, “Actionable cyber threat intelligence using knowledge graphs and large language models,” in *2024 IEEE European symposium on security and privacy workshops (EuroS&PW)*. IEEE, 2024, pp. 100–111.
- [4] Y. Cheng, O. Bajaber, S. A. Tsegai, D. Song, and P. Gao, “CTINexus: Automatic cyber threat intelligence knowledge graph construction using large language models,” in *Proceedings of the 10th IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2025, arXiv preprint arXiv:2410.21060.
- [5] L. Huang, Z. Liu, F. Shao, S. Ma, M. Zhang, Z. Chen, Y. Ye, and X. Xiao, “GRID: Graph representation of intelligence data for security text knowledge graph construction,” *arXiv preprint arXiv:2605.16714*, 2026.
- [6] M. T. Alam, D. Bhusal, L. Nguyen, and N. Rastogi, “CTIBench: A benchmark for evaluating LLMs in cyber threat intelligence,” in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024, arXiv preprint arXiv:2406.07599.

- [7] H. Ji, J. Yang, L. Chai, C. Wei, L. Yang, Y. Duan, Y. Wang, T. Sun, H. Guo, T. Li, C. Ren, and Z. Li, "SEVENLLM: Benchmarking, eliciting, and enhancing abilities of large language models in cyber threat intelligence," *arXiv preprint arXiv:2405.03446*, 2024.
- [8] D. Olszewski, A. Lu, C. Stillman, K. Warren, C. Kitroser, A. Pascual, D. Ukirde, K. Butler, and P. Traynor, "Get in Researchers; We're Measuring Reproducibility": A reproducibility study of machine learning papers in tier 1 security conferences," in *Proceedings of the 2023 ACM SIGSAC conference on computer and communications security*, 2023, pp. 3433–3459.
- [9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3971–3988.
- [10] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 729–746.
- [11] C. Wang, P. Zheng, J. Gui, C. Hua, and W. U. Hassan, "R+R: From claims to crashes: A systematic re-evaluation of graph-based network intrusion detection systems," in *Annual Computer Security Applications Conference (ACSAC)*, 2025.
- [12] M. T. Alam, D. Bhusal, and N. Rastogi, "R+R: Revisiting static feature-based android malware detection using machine learning," in *Annual Computer Security Applications Conference (ACSAC)*, 2025.
- [13] B. Cebere, J. L. B. Flueren, S. Sebastián, D. Plohmman, and C. Rossow, "Down to earth! guidelines for DGA-based malware detection," in *27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2024.
- [14] D. Schlette, F. Böhm, M. Caselli, and G. Pernul, "Measuring and visualizing cyber threat intelligence quality," *International Journal of Information Security*, vol. 20, no. 1, pp. 21–38, 2021.
- [15] L. Yang, M. Wang, and W. Lou, "An automated dynamic quality assessment method for cyber threat intelligence," *Computers & Security*, vol. 148, p. 104079, 2025.
- [16] T. Geras and T. Schreck, "The 'big beast to tackle': Practices in quality assurance for cyber threat intelligence," in *27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2024.
- [17] S. Wadhwa, S. Amir, and B. C. Wallace, "Revisiting relation extraction in the era of large language models," in *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*, 2023, pp. 15 566–15 589.
- [18] A. Swarup, T. Pan, R. Wilson, A. Bhandarkar, and D. Woodard, "LLM4RE: A data-centric feasibility study for relation extraction," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 6670–6691.
- [19] P. Jiang, J. Lin, Z. Wang, J. Sun, and J. Han, "Genres: Rethinking evaluation for generative relation extraction in the era of large language models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 2820–2837.
- [20] Q. Tan, L. Xu, L. Bing, H. T. Ng, and S. M. Aljunied, "Revisiting docred-addressing the false negative problem in relation extraction," in *Proceedings of the 2022 conference on empirical methods in natural language processing*, 2022, pp. 8472–8487.
- [21] Y. Fan, Y. Liu, Z. Yao, J. Yu, L. Hou, and J. Li, "Evaluating generative language models in information extraction as subjective question correction," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 2024, pp. 6409–6417.
- [22] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark nids datasets," in *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2024, pp. 658–675.
- [23] A. Ryan, S. S. Noor, M. Erfan, S. Mitra, S. Mittal, and M. R. Rahman, "Evaluating open-source LLMs for multi-label ATT&CK technique classification on CTI reports," *arXiv preprint arXiv:2606.18166*, 2026.
- [24] M. N. Haque, S. Hamer, B. Wroblewski, M. R. Rahman, and L. Williams, "Beyond single reports: Evaluating automated ATT&CK technique extraction in multi-report campaign settings," *arXiv preprint arXiv:2604.07470*, 2026.
- [25] M. R. Rahman, R. M. Hezaveh, and L. Williams, "What are the attackers doing now? automating cyberthreat intelligence extraction from text on pace with the changing threat landscape: A survey," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–36, 2023.
- [26] D. R. Arikkat, P. Vinod, R. R. KA, S. Nicolazzo, A. Nocera, and M. Conti, "Relation extraction techniques in cyber threat intelligence," in *International Conference on Applications of Natural Language to Information Systems*. Springer, 2024, pp. 348–363.
- [27] S. K. Lim, A. O. Muis, W. Lu, and C. H. Ong, "MalwareTextDB: A database for annotated malware articles," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1557–1567.
- [28] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, "TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of CTI sources," in *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC)*. ACM, 2017, pp. 103–115.
- [29] J. Zhao, Q. Yan, J. Li, M. Shao, Z. He, and B. Li, "TIMiner: Automatically extracting and analyzing categorized cyber threat intelligence from social data," *Computers & Security*, vol. 95, p. 101867, 2020.
- [30] I. Sarhan and M. Spruit, "Open-cykg: An open cyber threat intelligence knowledge graph," *Knowledge-based systems*, vol. 233, p. 107524, 2021.
- [31] Y. Guo, Z. Liu, C. Huang, J. Liu, W. Jing, Z. Wang, and Y. Wang, "CyberRel: Joint entity and relation extraction for cybersecurity concepts," in *Information and Communications Security: 23rd International Conference, ICICS 2021, Proceedings, Part I*, ser. Lecture Notes in Computer Science, vol. 12918. Springer, 2021, pp. 447–463.
- [32] H. Jo, Y. Lee, and S. Shin, "Vulcan: Automatic extraction and analysis of cyber threat intelligence from unstructured text," *Computers & Security*, vol. 120, p. 102763, 2022.
- [33] Y. Ren, Y. Xiao, Y. Zhou, Z. Zhang, and Z. Tian, "CSKG4APT: A cybersecurity knowledge graph for advanced persistent threat organization attribution," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5695–5709, 2023.
- [34] G. Wang, P. Liu, J. Huang, H. Bin, X. Wang, and H. Zhu, "KnowCTI: Knowledge-based cyber threat intelligence entity and relation extraction," *Computers & Security*, vol. 141, p. 103824, 2024.
- [35] K. Ahmed, S. K. Khurshid, and S. Hina, "Cyberentrel: Joint extraction of cyber entities and relations using deep learning," *Computers & Security*, vol. 136, p. 103579, 2024.
- [36] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang, "AttacKG+: Boosting attack knowledge graph construction with large language models," *Computers & Security*, vol. 150, p. 104220, 2025.
- [37] Y. Hu, F. Zou, J. Han, X. Sun, and Y. Wang, "LLM-TIKG: Threat intelligence knowledge graph construction utilizing large language model," *Computers & Security*, vol. 145, p. 103999, 2024.
- [38] L. Huang and X. Xiao, "CTIKG: LLM-powered knowledge graph construction from cyber threat intelligence," in *Proceedings of the First Conference on Language Modeling (COLM)*, 2024.
- [39] R. Zhong, Y. Chen, X. Yang, X. Zhong, and Z. He, "CTIExpert: Towards expert-level cyber threat intelligence extraction for constructing cybersecurity knowledge graphs," *Expert Systems with Applications*, p. 132921, 2026.
- [40] S. Kim, J. Kim, D. Kang, D. Kim, and I. Lee, "Schema-agnostic knowledge graph construction via hybrid ontology discovery for cyber threat intelligence," *arXiv preprint arXiv:2606.01208*, 2026.

- [41] M. A. Bouchiha and G. Blanc, "TACTIC-KG: Toward small agent teams for cyber threat intelligence knowledge graph construction," in *Proceedings of the 31st European Symposium on Research in Computer Security (ESORICS)*, 2026, to appear; preprint arXiv:2607.05001.
- [42] I. Mouiche and S. Saad, "Context-aware entity-relation extraction for threat intelligence knowledge graphs," *arXiv preprint arXiv:2605.15904*, 2026.
- [43] X. Yang, R. Zhong, Y. Chen, G. Peng, D. Yao, C. Chen, C. Wang, D. Zhang, Y. Zhou, and Z. Yang, "CTI-Thinker: an LLM-driven system for CTI knowledge graph construction and attack reasoning," *Cybersecurity*, vol. 9, no. 106, 2026.
- [44] B. Zhang and H. Soh, "Extract, define, canonicalize: An llm-based framework for knowledge graph construction," in *Proceedings of the 2024 conference on empirical methods in natural language processing*, 2024, pp. 9820–9836.
- [45] M. Buchel, T. Paladini, S. Longari, M. Carminati, S. Zanero, H. Binyamini, G. Engelberg, D. Klein, G. Guizzardi, M. Caselli, A. Continella, M. van Steen, A. Peter, and T. van Ede, "SoK: Automated TTP extraction from CTI reports—are we there yet?" in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, Aug. 2025, pp. 4621–4641. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/buechel>
- [46] Y. Cheng, C. Li, R. S. P. Basuki, Q. Cui, W. Ding, and P. Gao, "TTPrint: Evidence-grounded TTP extraction via diverge-then-converge verification," *arXiv preprint arXiv:2605.25836*, 2026.
- [47] L. Loconte, T. Hospedales, and C. Cornelio, "Better later than sooner: Neuro-symbolic knowledge graph construction via ontology-grounded post-extraction correction," *arXiv preprint arXiv:2605.29168*, 2026.
- [48] F. Valletta, G. Longo, E. Russo, and A. Merlo, "TRACE-CTI: Auditable post-extraction governance of TTP claims with knowledge graphs," *arXiv preprint arXiv:2607.24563*, 2026.
- [49] L. Yang, H. Zhu, J. Mu, and Q. Li, "KGAgent4CTI: Unlocking the power of LLM in threat intelligence," *Cybersecurity*, vol. 9, no. 1, p. 76, 2026.
- [50] B. Xue and L. Zou, "Knowledge graph quality management: a comprehensive survey," *IEEE transactions on knowledge and data engineering*, vol. 35, no. 5, pp. 4969–4988, 2022.
- [51] OpenRouter, "OpenRouter: A unified API for large language models," <https://openrouter.ai>, 2026, accessed 31 August 2026.
- [52] G. H. Chen, S. Chen, Z. Liu, F. Jiang, and B. Wang, "Humans or llms as the judge? a study on judgement bias," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 8301–8327.
- [53] M. T. R. Laskar, I. Jahan, E. Dolatabadi, C. Peng, E. Hoque, and J. X. Huang, "Improving automatic evaluation of large language models (llms) in biomedical relation extraction via llms-as-the-judge," in *Proceedings of the 63rd annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2025, pp. 25 483–25 497.
- [54] W. Cheng, T. Zhu, T. Chen, Q. Yuan, J. Ying, H. Li, C. Xiong, M. Li, M. Lv, and Y. Chen, "CRUcialG: Reconstruct integrated attack scenario graphs by cyber threat intelligence reports," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 6, pp. 6345–6360, 2025.
- [55] D. Olszewski, A. Lu, A. Crowder, N. Bennett, S. Layton, S. H. Varma Bhupathiraju, T. Tucker, S. Kalgutkar, H. Ver Helst, C. Stillman *et al.*, "Reproducibility in applied security conferences: An 11-year review on artifacts and evaluation committees," in *Proceedings of the 3rd ACM Conference on Reproducibility and Replicability*, 2025, pp. 96–107.

Appendix A. LLM Usage Disclosure

We disclose non-minor LLM usage in this work’s research pipeline. GPT-4o, accessed via OpenRouter, is the extraction backbone for the main full-benchmark CTIForge run and, in the matched-backbone comparison, for CTI-Nexus as well. The backbone-substitution study evaluates the six additional hosted and offline models listed in Section 6.5; Mistral Small 3.1 24B is used for the development-slice experiments documented in Appendix G. A sentence-transformer model (all-MiniLM-L6-v2) computes the embeddings used for entity alignment. No LLM was used to generate the claims, analyses, or prose of this paper.

Appendix B. Ontology Details

CTIForge uses a compact ontology of 14 entity types and 12 relation types so that the symbolic schema remains expressive enough for CTI while still being tractable for validation.

TABLE B1: CTIForge entity types.

Entity type	Description
ThreatActor	Groups, APTs, nation-state actors (e.g., APT29, Lazarus Group)
Malware	Malicious software families (e.g., LockBit, Emotet)
Tool	Legitimate software misused in attacks (e.g., Cobalt Strike, Mimikatz)
Campaign	Named attack campaigns (e.g., Operation Cobalt Kitty)
Vulnerability	CVEs and security flaws (e.g., CVE-2023-4966)
Technique	ATT&CK techniques (e.g., T1566 Phishing)
Tactic	ATT&CK tactics (e.g., Initial Access)
Organization	Companies, government agencies (e.g., Microsoft, CISA)
Location	Countries, regions (e.g., North Korea, Eastern Europe)
IOC	Indicators of compromise with subtype: IP, domain, URL, hash, email
File	Specific file names, hashes, paths (e.g., payload.dll)
Infrastructure	C2 servers, hosting infrastructure
Software	Legitimate software products (e.g., Microsoft Exchange)
Other	Entities not fitting above categories

TABLE B2: CTIForge relation types.

Relation type	Semantic scope
<i>uses</i>	Actor/malware uses tool, technique, or infrastructure
<i>targets</i>	Actor/campaign targets organization, sector, or location
<i>exploits</i>	Actor/malware exploits vulnerability or software
<i>delivers</i>	Campaign/malware delivers payload
<i>communicates_with</i>	Malware/tool communicates with IOC or C2
<i>drops</i>	Malware drops file or secondary payload
<i>attributed_to</i>	Malware/campaign attributed to threat actor
<i>associated_with</i>	General association (unconstrained)
<i>variant_of</i>	Malware family relationship
<i>located_in</i>	Organization/actor located in geography
<i>mitigated_by</i>	Vulnerability mitigated by tool, technique, or patch
<i>related_to</i>	Catch-all for relations not fitting above (unconstrained)

IOC subtypes are intentionally collapsed into one *IOC* type and overlapping relations are collapsed into a smaller inventory so that the validator can constrain outputs without forcing the LLM to discriminate overly fine-grained edge labels.

Algorithm C1 Symbolic Validation Pipeline

Require: Raw triple $t = (\text{subj}, s_t, \text{rel}, \text{obj}, o_t, \text{ev})$

Ensure: Validated triple t' with status ν

- 1: reject empty or too-short entities
- 2: repair identifiers and infer better types for *Other*
- 3: **if** $\neg \text{VALIDTYPEPAIR}(s_t, \text{rel}, o_t)$ **then**
- 4: **if** $\text{VALIDTYPEPAIR}(o_t, \text{rel}, s_t)$ **then**
- 5: swap arguments and mark as repaired
- 6: **else**
- 7: reject
- 8: **end if**
- 9: **end if**
- 10: reject forbidden self-loops
- 11: check evidence alignment and placeholders
- 12: validate ATT&CK identifiers
- 13: assign final status and log taxonomy action

Appendix C. Validation and Enhancements

Algorithm C1 gives the full check sequence applied to every raw triple. Order matters: identifier repair and type repair run before type-pair validation, so that a triple is only rejected once its recoverable defects have been corrected. Table C1 gives the constraint table the validation step consults.

The validator also applies regex-based normalization for CVEs, hashes, domains, and ATT&CK IDs; checks whether evidence text mentions the participating entities; lowers confidence for placeholders such as “the attacker”; and uses auto-swap repair for reversed but otherwise admissible relations.

C.1. Optional Neuro-Symbolic Enhancements

- *Link prediction*: propose connecting edges between disconnected subgraphs, subject to the same symbolic validator.
- *Embedding-based entity alignment*: merge same-type mentions with cosine similarity above a conservative threshold.
- *Saliency filtering*: keep only the highest-scoring triples per document using a symbolic ranker.
- *Supplement extraction*: a second LLM pass for missed triples; disabled because it over-generated non-gold triples.
- *Type-constraint-guided re-extraction*: ask about schema-licensed but missing pairs; disabled because valid additions often failed against selective gold annotations.
- *Multi-signal confidence re-scoring*: recalibrate confidence using symbolic signals; disabled because ranking quality did not align with gold-match status.

Appendix D. Extraction Prompt

The extraction prompt is a Jinja2 template with four blocks: an entity-type inventory, a relation-type inventory with disambiguation guidance, a JSON output schema, and

TABLE C1: Type-pair constraint table.

Relation	Valid subject types	Valid object types
<i>uses</i>	ThreatActor, Malware, Campaign, Tool	Malware, Tool, Technique, Infrastructure, IOC, Software, File
<i>targets</i>	ThreatActor, Malware, Campaign, Tool, Vulnerability	Organization, Location, Software, Infrastructure, IOC, File, Vulnerability, Other
<i>exploits</i>	ThreatActor, Malware, Campaign, Tool, Vulnerability	Vulnerability, Software, Infrastructure
<i>delivers</i>	ThreatActor, Malware, Campaign	Malware, File, Tool
<i>communicates_with</i>	Malware, Tool	IOC, Infrastructure
<i>drops</i>	Malware, ThreatActor, Campaign	Malware, File, Tool
<i>attributed_to</i>	Malware, Campaign, Tool, ThreatActor	ThreatActor, Organization, Location
<i>associated_with</i>	<i>any</i>	<i>any</i>
<i>variant_of</i>	Malware	Malware
<i>located_in</i>	Organization, ThreatActor, Infrastructure	Location
<i>mitigated_by</i>	Vulnerability, Organization, Software	Tool, Technique, Software, Organization
<i>related_to</i>	<i>any</i>	<i>any</i>

fifteen extraction rules. Few-shot examples, when enabled, are appended before the target text. Figure D1 gives its skeleton; the entity and relation inventories are those in Tables B1 and B2.

```

Extraction prompt — prompts/extraction.jinja

You are a cyber threat intelligence analyst. Extract
ALL structured knowledge triples from the following
CTI text.

## Entity Types
14 types, each with a one-line definition and 3–5 exemplars

## Relation Types
12 types; each states directionality and contrasts against the relation most often
confused with it

## Output Format
{"triples": [{"subject", "subject_type", "relation",
"object", "object_type", "evidence"}]}

## Rules
1. Extract 8–15 triples per chunk, prioritizing core threat relationships.
2. Every triple MUST have an evidence span copied from the text.
4. NEVER use related_to if ANY specific relation applies.
7. Direction matters: the subject performs the action, the object receives it.
15. If no triples can be extracted, return {"triples": []}.
{% if few_shot_examples %}
## Examples k retrieved exemplars
{% endif %}
## Text to Analyze
{{ text }}
```

Figure D1: Skeleton of the extraction prompt (prompts/extraction.jinja). Grey italics summarize blocks elided for space; rules are numbered as in the source.

Two design choices in this prompt bear on the results. First, the relation block defines each type *against the type it is most often confused with*, because the dominant strict-match failure is relation-label disagreement rather than entity identification (Section 6.5). Second, Rule 1 caps output at 8–15 triples per chunk: the extractor is asked to be selective at the point of generation rather than over-generating for a downstream filter to prune, which is consistent with the precision-first operating point and with the ablation result that supplementary extraction passes hurt precision.

Appendix E. Evaluation Protocol

Scoping. All reported metrics match predictions against gold *within a document*. Pooling a corpus into flat prediction and gold lists before matching permits a prediction from one report to be credited against another report’s annotations; measured on the 149-document benchmark this inflates true positives by 6.4% for CTIForge and 5.9% for CTI-Nexus, corresponding to roughly +0.03 F1 for both.

Name matching. Two entity mentions match if, after lowercasing and stripping leading determiners, they are string-equal, one contains the other, or their stemmed token sets overlap by at least 50% of the shorter set. Agent-noun equivalences (e.g. *attacker* / *attack*) are canonicalized before comparison.

Type equivalence. To avoid penalizing distinctions the gold annotations do not make consistently, three groups are treated as equivalent during type-aware matching: {Tool, Software, Malware}, {Technique, Tactic}, and {IOC, File, Infrastructure}.

Relation matching. The strict matcher requires identical relation labels after normalization into the 12-relation inventory. The relaxed matcher additionally accepts a fixed compatibility set that pairs the catch-all *related_to* with each specific relation, and admits inverse-perspective pairs such as (*exploits*, *mitigated_by*) and (*attributed_to*, *drops*). Matching proceeds in three passes—exact, name-soft with identical relation, then name-soft with compatible relation and a swap-aware check—each pass consuming gold triples greedily so that no gold triple is credited twice.

Reproducibility. Set iteration was replaced by sorted iteration in the subject–object matcher: per-process string-hash randomization otherwise changed greedy match order, producing a ± 0.0012 F1 band across runs on identical inputs. All reported figures are deterministic under repeated execution.

TABLE F1: Full module ablation across all seven extraction backbones, hosted above the rule and offline below it. Each backbone’s arms derive from one shared extraction pass. B+C+D is omitted: it is identical to B+C on every backbone to four decimals, which is the subject of Finding 6. Triples counts predictions emitted.

Backbone	Arm	P	R	Triplet F1	S-O F1	Triples
MiniMax-M3	B	0.4598	0.4293	0.4440	0.4928	1,703
	B+C	0.4723	0.4068	0.4371	0.4819	1,571
GPT-4o	B	0.5431	0.4764	0.5076	0.5485	1,600
	B+C	0.5509	0.4424	0.4907	0.5448	1,465
Haiku 4.5	B	0.4628	0.4227	0.4418	0.4865	1,666
	B+C	0.4652	0.3920	0.4255	0.4755	1,537
Sonnet 4.5	B	0.4777	0.3810	0.4239	0.4654	1,455
	B+C	0.4779	0.3553	0.4075	0.4604	1,356
Qwen2.5-7B	B	0.4788	0.3591	0.4104	0.4455	1,368
	B+C	0.4686	0.3065	0.3706	0.4269	1,193
Gemma-2-9B	B	0.5157	0.3695	0.4305	0.4606	1,307
	B+C	0.4931	0.3136	0.3834	0.4424	1,160
Mistral-7B	B	0.5045	0.3679	0.4255	0.4572	1,330
	B+C	0.4747	0.2982	0.3663	0.4384	1,146

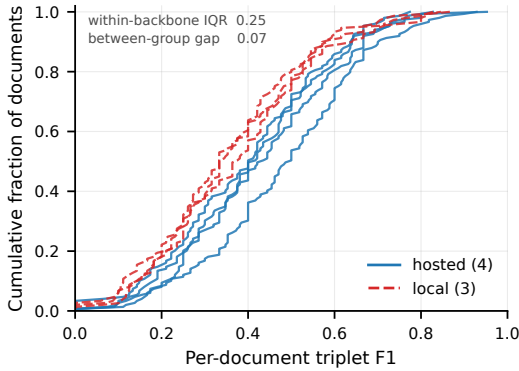


Figure F1: Per-document triplet F1 for seven backbones, colored by deployment configuration. Within-backbone spread exceeds the gap between group means, and the groups overlap substantially.

Appendix F. Additional Experimental Details

Appendix G. Data Availability

We release the code, configurations, matching protocols, and scripts needed to reproduce the reported tables: <https://github.com/sbhakim/CTIForge>. The third-party corpora and ten-system prediction cache remain available from their original releases [5].

Software and hardware environment

Experiments used Python 3.11. Hosted runs used temperature 0.1 through OpenRouter, while local runs used greedy decoding on an NVIDIA RTX 3090. The released configurations fix software versions and model identifiers:

TABLE G1: Datasets and resources.

Resource	Format	Scale	Role
CTI-Nexus annotations	JSON	149 reports	Primary document-level benchmark (1,824 gold triples)
CTIKG benchmark	CSV	255 sent.	Secondary sentence-level benchmark (693 gold triples)
MITRE ATT&CK	JSONL	1,076 rec.	Technique grounding and alias lookup
CVE database	JSONL	3,011 rec.	Vulnerability identifier validation
CWE/CAPEC	JSONL	1,957 rec.	Weakness and attack pattern reference
CTI blog corpus	JSONL	219 articles	Few-shot example retrieval

TABLE G2: Validator taxonomy by category, severity, and default action. Shaded categories are rejected rather than flagged; names match the identifiers in `error_taxonomy/error_log.jsonl`.

Category	Severity	Action	Description
<code>empty_field</code>	critical	reject	Empty subject, object, or relation
<code>impossible_type_pair</code>	critical	reject	Violates type-pair constraints
<code>self_loop</code>	critical	reject	Subject equals object
<code>hallucinated_entity</code>	high	reject	Entity absent from evidence
<code>malformed_identifier</code>	high	repair/reject	Malformed CVE, hash, or ID
<code>type_misassignment</code>	high	repair	Entity type inconsistent with name
<code>invalid_relation</code>	high	repair/reject	Relation not in schema
<code>generic_placeholder</code>	medium	lower conf.	Generic name (“the attacker”)
<code>missing_evidence</code>	medium	lower conf.	Evidence text empty
<code>duplicate_triple</code>	medium	reject	Duplicate of accepted triple
<code>confidence_lowered</code>	low	lower conf.	Partial evidence support
<code>other</code>	low	log	Uncategorized issue
<code>repaired_identifier</code>	info	repair	Auto-corrected format
<code>repaired_alias</code>	info	repair	Resolved to canonical name

pipeline entity alignment uses `all-MiniLM-L6-v2`, protocol embeddings use `all-MiniLM-L12-v2`, and entailment uses `facebook/bart-large-mnli`. All post-extraction stages are deterministic, including sorted matcher iteration.

Table G2 lists the fourteen declared taxonomy categories with their severity and default action; Table G3 gives the configuration used for every reported run. Both are stated in full so that a run can be reproduced without reading the source.

Appendix H. Validator Audit Statistics

Table H1 reports every validator action logged across both benchmark runs, by category and by the action taken. Counts are actions, not triples: a single triple may trigger more than one rule.

Seven of the fourteen declared categories fire on either benchmark. Three (*empty field*, *malformed identifier*, *repaired alias*) are implemented but guard conditions the output parser or alias tables resolve before validation is reached. Three (*duplicate entity*, *hallucinated relation*, *over-extraction*)

TABLE G3: Impact of configuration on the reported operating point: full hyperparameter set.

Parameter	Value
Primary LLM (149-doc run)	GPT-4o
Same-backbone head-to-head	GPT-4o via OpenRouter
Development slice	Mistral Small 3.1 24B via OpenRouter
Temperature	0.1
Max tokens	4,096
Retry attempts	3
Few-shot examples	3
Max chunk size	2,000 characters
Similarity threshold (τ)	0.85
Link prediction	enabled
Embedding entity alignment	enabled
Saliency filter	enabled (max 20 triples/doc)
Supplement extraction	disabled
Guided re-extraction	disabled
Confidence re-scoring	disabled

TABLE H1: Impact of each validation rule at benchmark scale: logged actions by category and outcome.

Category	CTI-Nexus	CTIKG	Default action
Confidence lowered	716	173	flag
Impossible type-pair	104	72	reject
Repaired identifier	104	23	repair
Type misassignment	75	162	flag
Generic placeholder	38	96	flag
Self-loop	17	12	reject
Hallucinated entity	15	12	flag
<i>Total actions</i>	<i>1,069</i>	<i>550</i>	
<i>Sources touched</i>	<i>146 / 149</i>	<i>193 / 255</i>	
Flagged	843 (78.9%)	443 (80.5%)	
Rejected	122 (11.4%)	84 (15.3%)	
Repaired	104 (9.7%)	23 (4.2%)	

are declared but unreachable in the current pipeline and are retained as specification for future work rather than reported as empirical categories.

Appendix I. Additional Results

This section collects supplementary results supporting the main claims.

I.1. Schema-Dependent Metrics

Table I2 reports definitional metrics against CTIForge’s own constraint table; they demonstrate contract enforcement, not superiority over a system that did not use that table. The semantic-validity margin is 27.7 points under this internal schema, compared with 13.3 under the external STIX 2.1 vocabulary in Table J1.

Appendix J. Matched-Backbone Comparison with CTI-Nexus

Table J1 compares both pipelines on all 149 documents using GPT-4o through the same endpoint. Because this comparison uses CTI-Nexus’s public matcher, its F1 values are

TABLE I1: Head-to-head comparison on the CTIKG benchmark (255 sentences, 693 gold triples) under the S-O pair (relaxed) match. Numbers in the “uniform” rows use the same matcher applied to both systems’ outputs; the “CTIKG (reported)” row restates the numbers published in the CTIKG paper, which use a different evaluation protocol.

System	Precision	Recall	F1	Predicted
<i>Uniform soft-match evaluation (both systems, same matcher)</i>				
CTIKG	0.5122	0.6955	0.5900	941
CTIForge (ours)	0.5722	0.4459	0.5012	540
<i>As reported by CTIKG (different protocol, reprinted for reference)</i>				
CTIKG (reported)	0.9189	0.8939	≈ 0.906	—

TABLE I2: Impact of scoring against the system’s own schema (definitional; not a cross-system comparison).

Metric (scored against SG’s constraint table)	SG	NX
Relation specificity	0.809	0.532
Entity-type specificity	0.980	0.795
Semantic validity	0.837	0.560
Schema constraint compliance	0.854	0.585
Evidence alignment rate	0.462	0.000
Cross-document entity fusion	0.107	0.085

controlled relative-comparison results rather than counterparts to the uniform-matcher values in Table 2.

As discussed in Section 6.4, the gold-matched rows describe a precision–recall trade-off under one protocol, not a protocol-independent ranking. The STIX 2.1 and evidence-presence rows are schema-independent contract differences.

Appendix K. Runtime Cost

On the identical 149-document benchmark, same backbone and same inference endpoint, CTIForge completes in **26.4 min** (10.6 s per document) against CTI-Nexus’s 38.7 min (15.6 s). Extraction is a single LLM call per chunk, with an optional second call for link prediction only when the document graph is disconnected, whereas CTI-Nexus runs a multi-phase prompting pipeline. The symbolic layer, canonicalization and fusion are deterministic and run locally with no model invocation. Section 6.5 shows the same pipeline runs on a free hosted backbone and offline on one consumer GPU, so cost is a property of the architecture rather than of the model tier.

Appendix L. CTIKG Comparison in Full

Table I1 compares CTIForge with CTIKG on the 255-sentence CTIKG benchmark (693 gold triples). The key methodological step is evaluating both systems’ outputs with a single matcher (substring containment and token-overlap ≥ 0.5 after normalization) rather than using each system’s own reporting protocol, so that differences in the reported numbers reflect extraction behavior rather than evaluator choice.

TABLE J1: Matched-backbone comparison on 149 documents using GPT-4o via OpenRouter. Gold-matched metrics depend on the stated matcher; schema-independent metrics do not use either system’s ontology. Constraint-table metrics appear separately in Appendix I. Winners are **bold**.

Metric	SG	NX	Winner
<i>Gold-matched (per-document scoping)</i>			
Triplet F1	0.4877	0.5449	NX
Triplet precision	0.5536	0.5100	SG
Triplet recall	0.4359	0.5850	NX
Documents won	57	85	NX
<i>Schema-independent (comparable across systems)</i>			
STIX 2.1 compliance	0.705	0.573	SG
coverage of output	0.782	0.368	—
Evidence presence	1.000	0.000	SG
Duplicate rate (↓)	0.022	0.055	SG
<i>Normalization artifacts (↓ is cleaner)</i>			
Generic relation rate	0.191	0.469	—
Other entity rate	0.021	0.205	—
Triples emitted / gold	1,436 / 1,824	2,092 / 1,824	—
Audit trail	1,359 val., 77 rep.	none	SG

The resulting precision–recall trade-off and the gap between uniform and originally reported scores are interpreted in Section 6.4; the table is retained here to expose the underlying values.